

PPLP: Privacy-Preserving Link Prediction

Kaleb Kim, Wiam Skakri, Carson Whitehouse, Jonah Lorenzo
Kent Manion, and Aidan Bugayong
Case Western Reserve University
Cleveland, Ohio, USA

Abstract

Distributed link prediction over multiple graphs can improve accuracy but raises privacy concerns. We describe a Python library for *privacy-preserving link prediction* (PPLP): two parties compute link-prediction scores over the union of their graphs without revealing graph structure. The library implements the protocol of Demirag et al. [2] on top of OpenMined PSI [6] (substituted for the originally planned ultra-fast PSI of Ling et al. [5] for portability), supports Common Neighbors and Jaccard, and is exposed as a Python API, a CLI server, and a Streamlit dashboard. The principal design choice is to recast the paper’s symmetric peer protocol as an asymmetric server–client service, enabling a data-as-a-service deployment but introducing a new threat surface: adaptive queries against a fixed server graph. We present microbenchmarks (≤ 0.6 s per query at neighborhood sizes of 1 000), a sequence-level breakdown of the HTTP API, an end-to-end demonstration over four datasets, and the residual threat model.

Keywords

link prediction, private set intersection, homomorphic encryption, distributed graphs, privacy

1 Problem Statement and Literature Search

1.1 Link Prediction

Link prediction aims to discover unobserved or latent connections between nodes in a graph [4]. Given a snapshot of a network at time t , the goal is to predict which edges will appear by a future time t' using structural information (e.g., proximity measures or graph neural networks). Applications include social networks, e-commerce, telecommunications, fraud and financial crime detection, and bioinformatics—see Section 1.3.

Link prediction is typically performed on a single local graph. Distributed link prediction considers the setting where two or more parties hold related graphs (e.g., overlapping node sets or the same domain). Merging these graphs can yield more accurate predictions, but sharing raw graph data raises privacy risks: identity disclosure, link disclosure, and attribute disclosure. We therefore focus on privacy-preserving link prediction (PPLP): multiple data holders collaboratively compute link-prediction scores over their combined

graph *without* revealing their private graph structure to each other (Figure 1).

Contributions. This work delivers (i) an open-source Python library implementing the Demirag et al. [2] two-party PPLP protocol, accessible through a Python API, a CLI server, and a Streamlit dashboard (Section 3); (ii) a recasting of the symmetric peer protocol as an asymmetric server–client service that supports a data-as-a-service deployment (Section 3.4); (iii) microbenchmarks of latency and wire payload across neighborhood sizes 10–1 000 (Section 3.8); and (iv) a characterization of the adaptive-query threat surface that the service deployment introduces and a sketch of the layered defenses required to close it (Sections 4.1 and 8.1).

1.2 Related Work

Liben-Nowell and Kleinberg [4] formalize the link-prediction problem using only network topology (no node attributes) and conduct a study of proximity-based measures on co-authorship networks. Their analysis establishes common neighbors, Jaccard, and Adamic–Adar as foundational measures that significantly outperform random baselines on co-authorship data. This topology framing allows for a set-based approach that enables compatibility with Private Set Intersection (PSI).

Demirag et al. [2] give a two-party protocol in which each party holds a graph; they compute *common neighbor* counts over the union of both graphs using a notably small number of private set intersection (PSI) and additively homomorphic encryption calls. The common-neighbor count is expressed as local terms plus crossover terms minus overlaps such that only intersection sizes rather than neighbor sets are revealed. They also discuss a heavier homomorphic variant to reduce intermediate leakage where neither party can learn the intermediate intersection sizes before the final output is assembled. Their threat model assumes semi-honest (honest-but-curious) adversaries and no trusted third party, which is realistic for competitive or regulated settings such as cross-institutional healthcare.

PSI is the core cryptographic primitive enabling the PPLP protocol, as it lets two parties compute the intersection of their sets while revealing only the result. Chen et al. [1] give a high-performance PSI protocol from fully homomorphic encryption (FHE) with communication complexity $O(N_{\text{small}} \cdot \log N_{\text{large}})$. Lattice-based FHE [3] made such protocols practical by bootstrapping and resolving the noise problem with a decryption circuit. Ling et al. [5] propose an ultra-fast PSI from efficient Oblivious Key-Value Stores (OKVS) and Vector Oblivious Linear Evaluation (VOLE), with $O(n)$ communication and low encoding redundancy (on the order of 1%), making it significantly faster in practice for large sets than FHE-based PSI. We originally planned to base our Python library on this approach for

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CSDS 356/456, Cleveland, OH

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/XXXXXXX.XXXXXXX>

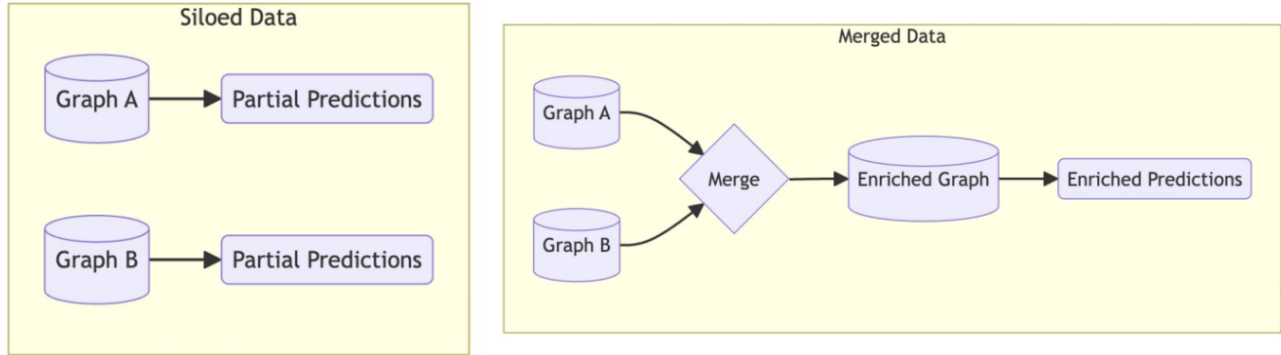


Figure 1: Siloed vs. merged data. Each party independently produces only partial predictions (left), whereas merging graphs yields an enriched, more accurate result (right), at the cost of privacy without PPLP.

its linear communication cost; Section 3.3 discusses the substitution we ultimately made for portability.

1.3 Motivating Use Cases

Applications span several domains, each presenting distinct graph structures and privacy considerations.

Social networks: PPLP powers friend recommendations by surfacing latent connections between users across a network snapshot. Sharing edge data across multiple networks risks re-identifying pseudonymous users through structural patterns.

E-commerce: PPLP drives product recommendation by predicting new edges in a bipartite user-product graph built from purchase and interaction history. Multiple retailers may hold overlapping customer bases, but sharing purchase graphs exposes proprietary customer behavior data.

Telecommunications: PPLP enables targeted advertising using call-/message patterns among users; such graphs disclose routines, so pooling them across carriers risks attribute-/link leakage.

Bioinformatics: PPLP merges EHR-linked clustering graphs for research on diagnosis, outbreaks, or drug interactions without requiring either site to disclose full subgraphs outside protocol-authorized statistics.

Financial networks: PPLP supports fraud detection by identifying suspicious latent connections between accounts at different institutions through intermediary accounts—an indicator studied in anti-money laundering. Raw graph sharing is largely barred by regulation; PPLP would let cooperating institutions tighten risk signals without pooling raw transactional graphs centrally.

2 Planned Methods

The subsections that follow restate the methodology and threat model from our midterm submission; tense and scope are preserved unchanged for consistency with that document. Section 3 describes the delivered system and how it maps to this plan.

2.1 Link-Prediction Measures

We will support at least one of the following proximity-based measures, which can be expressed using set operations over neighbor sets and thus combined with PSI.

- **Common Neighbors (CN):** For nodes u and v , $CN(u, v) = |N(u) \cap N(v)|$. More common neighbors imply higher link likelihood.
- **Jaccard:** $\frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|}$, which accounts for the relative overlap.
- **Adamic-Adar (AA):** $\sum_{w \in N(u) \cap N(v)} \frac{1}{\log |N(w)|}$, emphasizing common neighbors with lower degree.

2.2 Threat Model

We assume a two-party, semi-honest (honest-but-curious) adversary model. Both Alice and Bob execute the protocol steps faithfully; they do not deviate, send malformed inputs, or abort early. However, each party attempts to infer as much as possible about the other’s private graph from the protocol messages they legitimately receive. Each party enters the protocol knowing its own graph in full, the set of candidate node pairs for which link scores are being computed, and the public protocol specification.

The PSI calls expose only the sizes of intersection sets (e.g., $|I_{A+B}|$ and the crossover intersection sizes), not the sets themselves. The protocol aggregates these into each candidate pair’s common-neighbor total; neither party learns the other’s neighbor list as identifiable sets—only cardinality statistics leave PSI calls. Alice must not learn $N_B(u)$ for any node u , and Bob must not learn $N_A(u)$.

The basic protocol leaks intermediate intersection sizes, which may allow inferences about the other party’s graph degree distribution. Demirag et al. propose a heavier homomorphic variant that eliminates even this leakage; we plan to explore this as an extension. Malicious adversaries, party collusion, side-channel attacks, and settings with more than two parties are out of scope for the initial prototype.

2.3 Two-Party Common Neighbors via PSI

Following Demirag et al., for two parties Alice (graph G_A) and Bob (graph G_B), the common-neighbor count for a candidate pair (A, E)

can be decomposed into the following local and crossover terms. Figure 2 shows a concrete example where Alice holds G_A over nodes $\{A, B, C, D, E, G, H\}$ and Bob holds G_B over nodes $\{A, B, C, D, E, K, F\}$, with the two graphs sharing the node subset $\{A, B, C, D, E\}$.

- Local intersections: $I_A(A, E) = N_A(A) \cap N_A(E)$ and $I_B(A, E) = N_B(A) \cap N_B(E)$ (each party computes locally).
- Crossover terms: e.g., $N_A(A) \setminus I_A$ with $N_B(E) \setminus I_B$, etc., which require PSI so that only intersection sizes (or masked sets) are revealed.

The total $|\text{CN}(A, E)|$ is then obtained as $|I_A| + |I_B| - |I_A \cap I_B|$ plus the contributions from the crossover PSI results (e.g., $|I_{A+B}|$ and the sizes of the privately computed crossover intersections). The protocol uses a small, fixed number of PSI calls per candidate pair. Figures 3 and 4 walk through this computation on the running example.

2.4 Two-Party Node Cardinalities via PSI

Similar to Common Neighbors, two parties can find out the combined cardinalities for common nodes between the parties. Assume again that Alice (graph G_A) and Bob (graph G_B) first perform PSI on the node sets of their graphs to determine which nodes they share, then compute combined cardinalities for those shared nodes. Below are the steps to determine the combined cardinality of node A :

- Local Cardinalities: $N_A(A)$ and $N_B(A)$ (each party computes locally).
- Intersection Term: $N_A(A) \cap N_B(A)$, which requires PSI so that only the intersection size (or masked set) is revealed.

The total $|N_{A+B}(A)|$ is then obtained as $|N_A(A)| + |N_B(A)| - |N_A(A) \cap N_B(A)|$. The protocol uses a single PSI call per node, although an additional PSI call is required to determine which nodes are available to both parties.

2.5 Privacy Primitives

- **Private Set Intersection (PSI):** Core primitive to compute intersections (or their sizes) over neighbor sets without revealing the sets themselves. We plan to base our implementation on the ultra-fast PSI of Ling et al. (OKVS + VOLE) for $O(n)$ communication and low bandwidth.
- **Homomorphic encryption (HE):** Used in the protocol to compute on encrypted neighbor sets (e.g., for the polynomial-based PSI of Chen et al. or as in Demirag et al.); FHE allows operations on ciphertexts without decryption.

2.6 System Architecture

- **Core backend (C++):** High-performance PSI and cryptographic primitives (e.g., leveraging an existing implementation such as ShallMate/fastpsi for the Ling et al. protocol).
- **Python bindings:** Pybind11 or Cython to expose the backend to Python, so that data scientists can run PPLP without writing low-level code.
- **Optional front-end:** A web dashboard (e.g., React or Streamlit) for uploading graph snapshots and visualizing predicted links.

3 Proposed Solution

The remainder of this report describes the system we ultimately built. Section 3.1 relates it to the methods, next steps, and expected output we proposed at midterm; Sections 3.3–3.8 describe the as-built system and its measured behavior.

3.1 Midterm Plan vs. Delivered

At midterm we planned a C++ PSI core (Ling et al. [5]) wrapped by Python bindings, a two-party CN protocol following Demirag et al. [2], multi-machine endpoints, and an optional web UI for demos; the expected deliverable was a documented Python library supporting at least Common Neighbors, a worked example, and an optional dashboard. We delivered all of these except Adamic–Adar (Section 7 explains why) and substituted OpenMined PSI for fastpsi on portability grounds (Section 3.3). Two-party Common Neighbors and Jaccard, multi-machine deployment via FastAPI and a public tunnel, and the Streamlit dashboard all shipped (Sections 3.4–3.8, 5).

3.2 Overview

The delivered system is a Python library installable via `uv` or `pip`. It contains a graph data structure, the protocol orchestration code, a FastAPI server for Party 2, an HTTP client for Party 1, and a Streamlit dashboard. A `pplp-server` CLI entry point launches the server with optional tunnel exposure.

3.3 PSI Backend: From fastpsi to OpenMined PSI

The midterm plan called for a C++ core leveraging fastpsi (Ling et al. [5]). We ruled this out for three reasons: fastpsi is Linux-only and requires AVX-512, which our development hardware (Apple Silicon, older x86) lacks; it builds with Bazel + YACL and ships no Python bindings, requiring a hand-authored binding layer; and it has no PyPI distribution, forcing every downstream user to compile from source.

We instead use OpenMined PSI [6], an ECDH-based PSI library with a C++ backend, Python bindings, and prebuilt wheels for macOS and Linux. Cardinality-only mode suppresses revelation of intersecting elements and exposes $|A \cap B|$ alone—exactly what the Demirag/Ayday protocol requires—and we use Golomb Coded Sets (GCS) for the response to minimize bandwidth. The substitution forfeits OKVS+VOLE’s asymptotic advantage but yields a pip-installable library, which we judged the more important property for a class deliverable.

3.4 System Architecture: Recasting the Protocol as a Service

The midterm architecture (Section 2.6) described a software stack but not a deployment model. The Demirag/Ayday paper presents the protocol as a symmetric two-party rendezvous: two peers come together for one computation and disband. This is a clean cryptographic abstraction but a poor deployment story; it requires both parties to be online, mutually reachable, and willing to perform ad-hoc setup per query.

We therefore recast the protocol as an asymmetric server–client architecture. Party 2, the data holder, runs a long-lived FastAPI

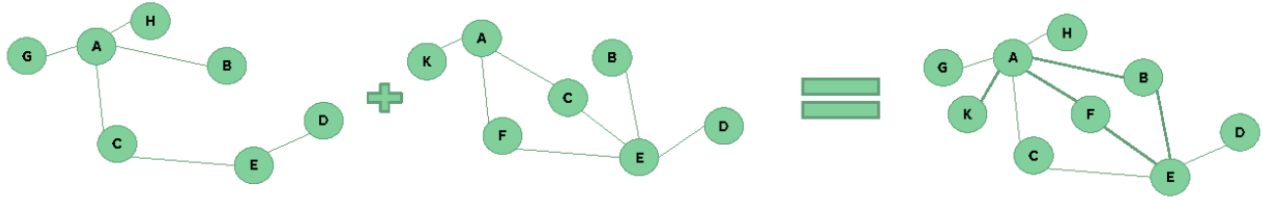


Figure 2: Two-party graph union. Alice's graph G_A (left) and Bob's graph G_B (center) share overlapping node set $\{A, B, C, D, E\}$; their union (right) contains additional nodes G, H from Alice and K, F from Bob.

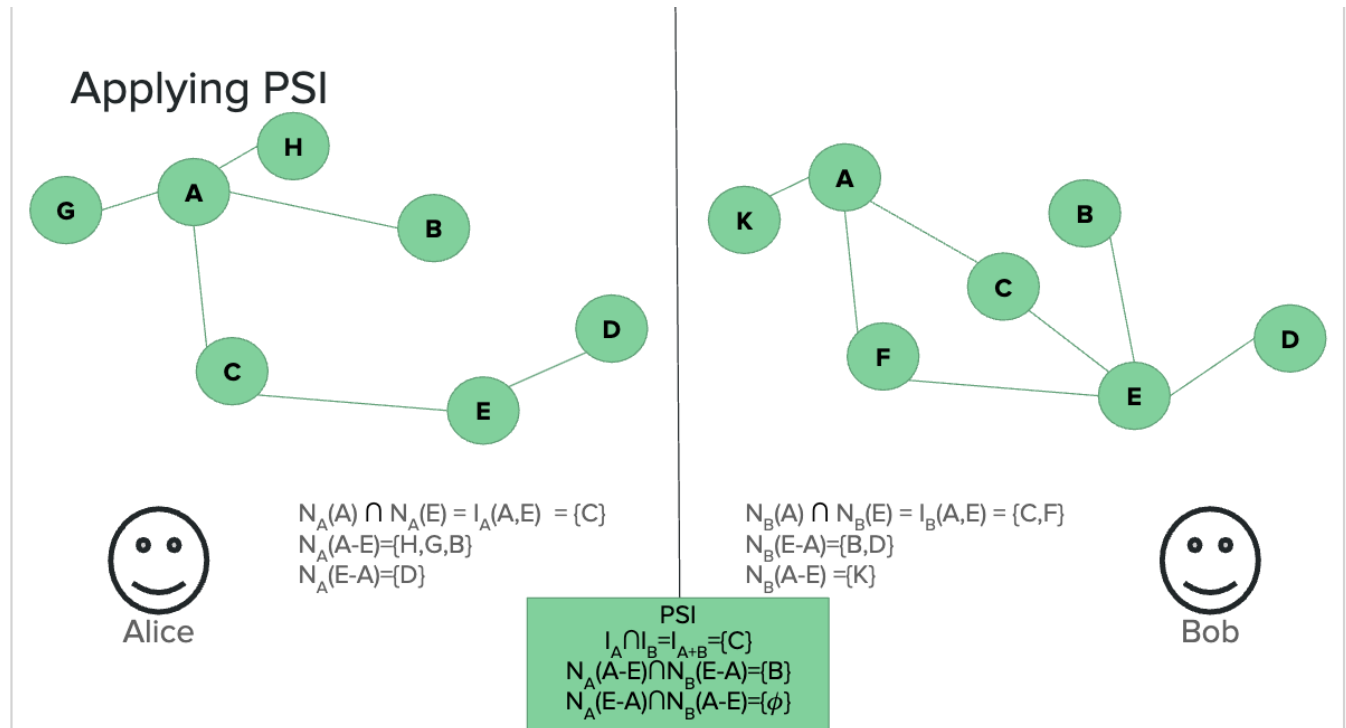


Figure 3: Applying PSI for candidate pair (A, E) . Alice and Bob first compute local intersections I_A and I_B independently. Three PSI calls then privately resolve the crossover terms: $I_A \cap I_B = \{C\}$, $N_A(A-E) \cap N_B(E-A) = \{B\}$, and $N_A(E-A) \cap N_B(A-E) = \emptyset$.

server preloaded with its graph. Party 1 is a stateless client that connects, opens a session, and drives the protocol to completion. This changes the deployment model in four ways:

- **Data-as-a-service.** The data owner exposes query access without releasing the graph, enabling a pay-per-query API. The symmetric peer protocol offers no such interface.
- **Operationally one-sided.** Only the server needs a stable network identity. The client (a notebook, CI job, or Streamlit session) needs only the server URL and an edge list.
- **Session-scoped state.** Each candidate pair owns a server-side session holding three (CN) or five (Jaccard) PSI server objects with fresh ECDH keys. The session is the natural unit for rate limiting, audit logging, and quota enforcement.

- **Cryptographic guarantees unchanged.** The protocol is the one analyzed in Section 2.2: only Party 1 learns the final score, only intersection sizes leave Party 2, and PSI keys are fresh per call.

The cost of recasting is a new threat surface. A long-lived server is by construction queryable; a peer that walks away cannot be probed, a service can. This shifts the open problem from whether a single query is private (which the paper answers) to whether a *stream* of queries is private (which it does not). Section 4.1 and Section 8.1 address this gap.

The HTTP API exposes four endpoints. GET `/health` is a liveness check. POST `/prepare` accepts (x, y) and a measure, computes Party 2's local intersection, reduces its neighbor sets, stores the per-call sets in a session, and returns the local intersection

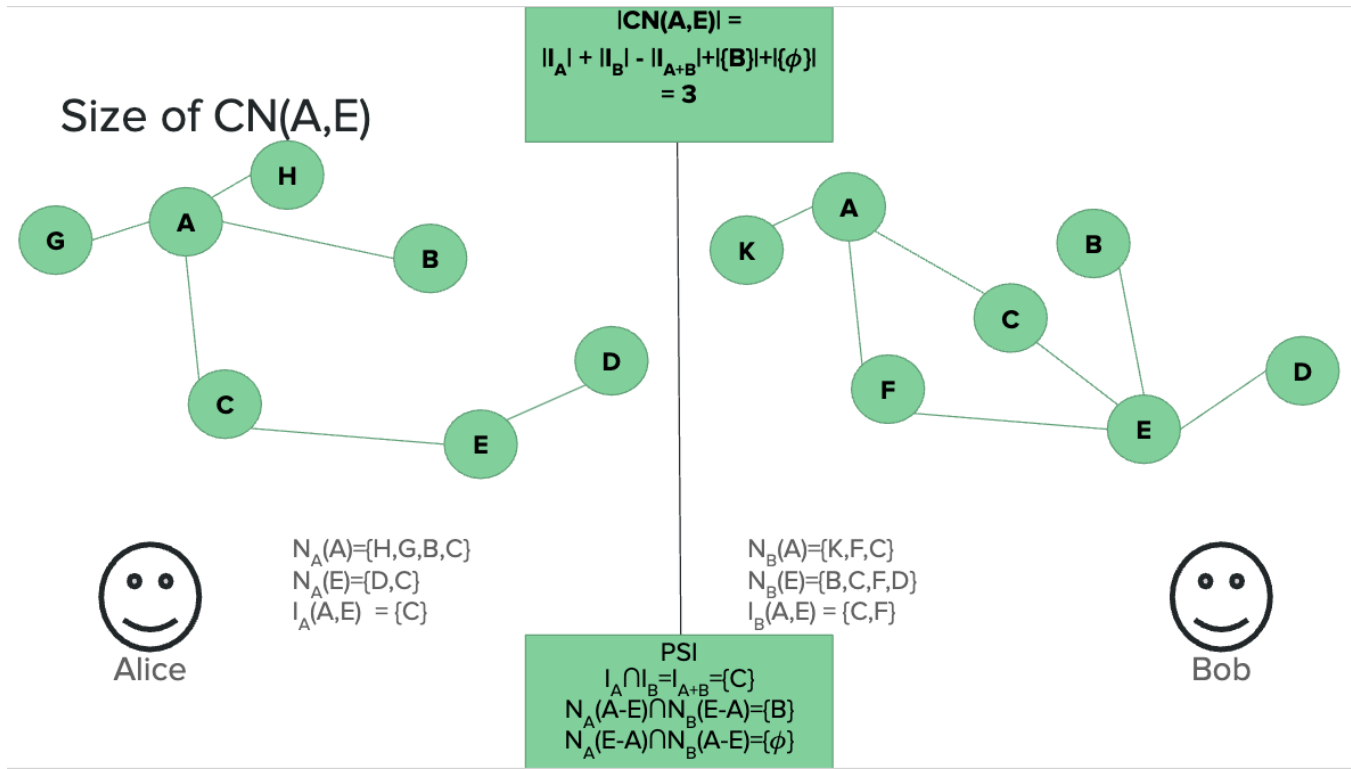


Figure 4: Computing $|CN(A,E)|$. Combining local terms $|I_A|=1$ and $|I_B|=2$ with the PSI-computed crossover results yields $|CN(A,E)| = 1 + 2 - 1 + 1 + 0 = 3$.

size, a session ID, and (for Jaccard) the two degree sizes. `POST /psi/{sid}/setup` creates an ECDH PSI server with a fresh key for one named call and returns its base64-encoded setup message. `POST /psi/{sid}/respond` processes Party 1's base64-encoded request and returns the response, from which Party 1 extracts the cardinality. Figure 5 shows one full Common Neighbors query.

3.5 Two-Party Jaccard via PSI

Jaccard is a direct extension of the CN protocol. The numerator is $CN(x, y)$; each union size is $|N_1(x) \cup N_2(x)| = |N_1(x)| + |N_2(x)| - |N_1(x) \cap N_2(x)|$, and analogously for y , giving

$$J(x, y) = \frac{CN(x, y)}{D_x + D_y - CN(x, y)},$$

$$D_x = |N_1(x) \cup N_2(x)|, \quad D_y = |N_1(y) \cup N_2(y)|.$$

Each union adds one PSI call (the degree overlap), and Party 2 must reveal $|N_2(x)|$ and $|N_2(y)|$ in `/prepare`. Total cost: 5 PSI calls per query and a 2-integer degree leak.

3.6 Network Layer and Wire Format

Consumer machines on different networks generally cannot reach each other directly because of NAT, firewalls, and lack of public IPs. We address this with *localtunnel*, a Node.js tool that exposes a local port at a randomly assigned `*.localtunnel.net` subdomain. The Streamlit dashboard launches the uvicorn server and tunnel together and prints the public URL; the CLI server (`pplp-server`) offers a parallel

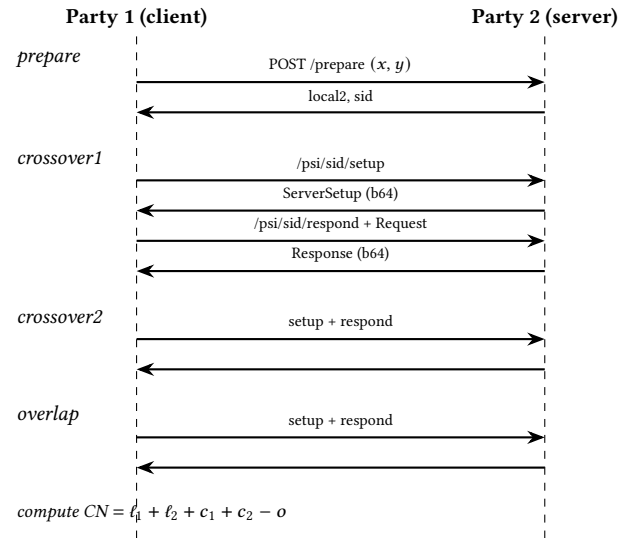


Figure 5: HTTP message flow for one Common Neighbors query (3 PSI calls = 6 round trips after `/prepare`). Jaccard adds two further *degree_x* and *degree_y* PSI calls.

ngrok-based path. Localtunnel interposes a one-time splash page, which the client bypasses with the vendor-recommended HTTP

header (Bypass-Tunnel-Reminder). No firewall configuration is required on either side.

OpenMined PSI returns its setup, request, and response messages as protocol buffer objects. JSON cannot carry binary directly, so each is base64-encoded on the wire ($\approx 33\%$ overhead). Because PSI message sizes are dominated by GCS-compressed cryptographic material, this overhead is small in absolute terms (Section 3.8, Table 1).

3.7 Frontend and Validation

The Streamlit dashboard provides the web frontend anticipated at midterm. Each party uploads a CSV edge list, selects a role, and either starts the server or supplies a server URL. The same FastAPI endpoints back the dashboard, the Python API, and the test suite. Figure 6 shows Party 2 (server mode) exposing a localtunnel URL and Party 1 (client mode) configuring a tunnel URL plus a CN/Jaccard query with a numerical result visible in-browser.

Correctness is verified against an in-process plaintext baseline. The test suite covers the graph data structure, the local PSI primitive, local CN and Jaccard, the FastAPI server, the HTTP client, and end-to-end remote computation on synthetic graphs. The paper’s worked example (Figure 4, $|\text{CN}(A, E)| = 3$) is one of the test cases, and remote and local results agree.

3.8 Microbenchmarks

Table 1 reports timing and wire-payload measurements for synthetic graphs in which both candidate nodes have neighborhoods of the listed size. Each row is the median of multiple trials; the FastAPI test client is in-process, so network latency is excluded. Three observations follow:

- Local PSI cardinality scales roughly linearly with neighborhood size (3.2 ms at 10 neighbors $\rightarrow$ 161 ms at 1 000).
- Remote CN takes $\approx 3\times$ a single PSI call (3 PSI calls); remote Jaccard takes $\approx 5\times$ (5 PSI calls). The HTTP layer adds 5–15 ms of constant overhead per query.
- Wire payload is dominated by the PSI *response* (GCS-encoded ECDH curve points). At a neighborhood size of 1 000 the average response body is ≈ 47 kB; the full CN query transfers ≈ 150 kB inclusive of base64 overhead. /prepare bodies are constant (≈ 113 B).

A real wide-area deployment will dominate these numbers with round-trip latency: each CN query costs 7 round trips (1 /prepare + $3\times$ setup + $3\times$ respond), and Jaccard costs 11. At a typical 50 ms RTT, that is 350 ms of network time on top of the figures in Table 1.

4 Threat Model in Practice

The semi-honest analysis of Section 2.2 carries over for any single run. Recasting the protocol as a long-lived service (Section 3.4) introduces two new threat surfaces the midterm did not consider: adaptive queries against a fixed server graph, and metadata observable through the public tunnel.

Table 1: Per-query latency and wire payload as a function of per-side neighborhood size $|N(x)| = |N(y)|$. Latencies in milliseconds (median of 3–5 trials), payloads in bytes per HTTP exchange. Apple Silicon, in-process FastAPI test client.

$ N $	Latency (ms)			Wire (B/call)		
	PSI	CN _{rem}	Jac _{rem}	prep	setup	resp
10	3.2	20.8	23.3	112	84	643
50	9.4	30.0	61.5	113	152	2 854
100	18.7	37.9	94.0	113	216	4 750
500	82.0	140.0	287.6	114	870	23 667
1 000	160.5	246.0	567.3	114	1 782	47 283

Abbreviations. Columns *prep*, *setup*, and *resp* are mean byte payloads for the /prepare handshake, each PSI server’s setup, and subsequent /respond exchanges (Section 3.4).

4.1 Adaptive Queries: The Cost of Being a Service

Demirag et al. [2] prove that any single run leaks only intersection sizes; the analysis says nothing about what an adversary learns from a *stream* of carefully chosen queries against a fixed graph. A long-lived server implicitly grants this capability. Three attacks lie outside the prototype’s defended threat model:

- **Iterative graph reconstruction.** A malicious client mutates its local graph between queries and observes score changes. Each query reveals one integer, but a few hundred well-chosen queries can recover server-side neighborhoods with high fidelity. The semi-honest analysis does not cover this case: each call is protocol-faithful, but inputs are chosen adaptively across calls.
- **Pair-query oracle.** A malicious client synthesizes a one-edge local graph connecting two server-side node identifiers it cares about. The CN score is then dominated by the server’s contribution and effectively reveals structural information about neighborhoods on Party 2—an unintended channel beyond the cardinality-only PSI release.
- **Malicious-server dual.** A server may substitute a fully connected (or null) graph between Party 1’s queries; repeated queries against such manufactured graphs let the server reverse-engineer Party 1’s local graph from responses Party 1 trusts.

None of these breaks the PSI primitive; they exploit the fact that a queryable service is a function the adversary can invoke repeatedly, and the protocol guarantees nothing about query streams. Defending against adaptive queries is the central open problem our architecture introduces; Section 8.1 sketches a layered defense.

4.2 Tunnel-Layer Metadata

Routing the server’s public reachability through localtunnel introduces a third party that sees every connection. The PSI payload is cryptographically opaque, but four side channels remain. Source IP addresses identify clients (defeating any naive Sybil-resistance scheme that relies only on session state). Request *timing*—in particular the spacing between /prepare and the trailing six (or ten) PSI calls—reliably distinguishes CN from Jaccard queries. Request

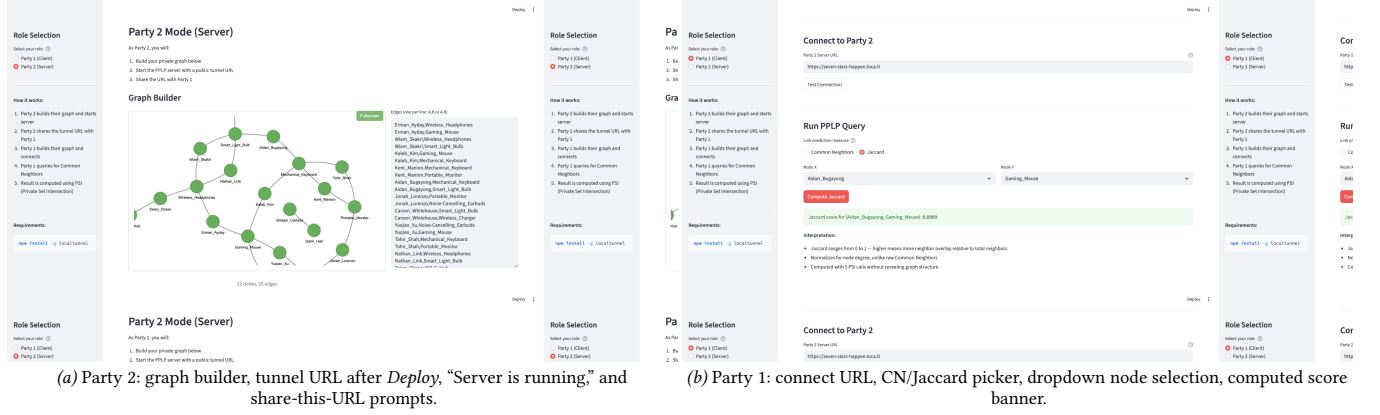


Figure 6: Streamlit dashboard excerpts. Panel (a) is Party 2 after deployment; Panel (b) is Party 1 running a scored query against the server’s public URL.

sizes leak the per-call PSI request length, which scales with $|N_1(x)|$ and $|N_1(y)|$ (Table 1); the tunnel can therefore estimate the client’s neighborhood sizes per query, which is information the protocol does not authorize. Finally, the JSON envelope itself (endpoint paths, session IDs) is plaintext on the tunnel side. A production deployment should terminate TLS only at the data-holder’s own infrastructure, or use a trusted reverse proxy, rather than rely on a free third-party tunnel for transport.

4.3 Other Out-of-Scope Adversaries

Collusion among more than two parties, side-channel attacks on the cryptographic library, and active network adversaries (man-in-the-middle on the tunnel beyond the metadata above) are out of scope. A production deployment would additionally need to handle denial-of-service against the in-memory session store and the cost asymmetry of forcing the server to do PSI work per client request.

5 Demo and Use Cases

We demonstrated the system end-to-end with one server graph and three independent client graphs, mirroring the cross-domain scenarios from Section 1. The server graph is a small social network of “co-friend” relationships among the project team and instructors, playing the role of a data-rich provider that wants to monetize query access. Each client holds a bipartite graph drawn from a different domain: Client 1 a user–product retail graph, Client 2 a patient–disease EHR-style graph, and Client 3 a user–show entertainment graph.

Across all four configurations the remote CN and Jaccard outputs agreed exactly with an in-process plaintext baseline computed over the unioned graph, confirming the protocol implementation is correct. The queries fall into four observable categories that we name here and reuse in Table 2:

- **Productive (P).** At least one of x or y is well represented in the server graph, both graphs contain a common node type (e.g., both contain person nodes), and x and y are not already directly connected.

Table 2: Representative queries from the demo, classified by the categories above (P = productive, N = no common attribute, L = low server presence, S = sparse client data). All remote results matched the plaintext baseline.

Client	Candidate pair	Type	CN	Jac.
Retail	(Kaleb_Kim, Wireless_Headphones)	P	high	high
Retail	(Wiam_Skakri, Mechanical_Keyboard)	P	high	high
Retail	(Darin_Hall, Wireless_Headphones)	L	0	0
Retail	(Wireless_Headphones, Mech_Keyboard)	N	0	0
EHR	(Erman_Ayday, Goated_Teaching_Disorder)	S	0	0
EHR	(Carson_Whitehouse, Insomnia)	S	0	0
EHR	(Flu, Covid)	N	0	0
Shows	(Wiam_Skakri, Penguins_of_Madagascar)	P	high	high
Shows	(Darin_Hall, The_Barbie_Show)	L	0	0

- **No common attribute (N).** Neither x nor y appears in the server graph, or both belong to a node type the server does not store. The query degenerates to local link prediction.
- **Low server presence (L).** A queried node exists on the server but has very few server-side connections, so the cross-graph contribution is dominated by noise.
- **Sparse client data (S).** The client side itself has too few edges per node for any cross-graph statistic to be meaningful (the EHR client, where most diseases appeared 1–3 times, was the canonical case).

The classification is a useful diagnostic for clients: only category P queries justify the cross-graph cost: the other three return scores indistinguishable from local link prediction and can be filtered out before they consume server budget. Section 7 discusses the lack of a built-in candidate generator that would automate this filtering.

6 Design Tradeoffs

Five tradeoffs shaped the system.

Privacy vs. efficiency. The heavier PSI pipeline described by Demirag et al. [2] hides PSI intermediates from Party 1. We nevertheless shipped the lighter protocol, trading additional bounded structural leakage directly against avoided FHE cost and ciphertext noise bookkeeping.

Accuracy vs. efficiency. Jaccard yields a more meaningful 0–1 score than raw CN counts but costs 5 PSI calls and reveals two degree sizes (vs. 3 calls, no extra leakage for CN). We expose both and let the application choose.

Memory vs. persistence. Server-side sessions live in an in-process Python dict with no eviction. Sufficient for a single-process demo; production needs an external store (e.g., Redis) with a TTL (Section 7).

Neighbor counting vs. weighting. CN and Jaccard treat all neighbors equally and map cleanly onto cardinality PSI. Adamic–Adar and Katz weight neighbors by per-node statistics that cardinality PSI does not expose, and need a different primitive (Section 7).

Library portability vs. raw PSI speed. Section 3.3: we picked ECDH PSI for the pip-installable library. Table 1 confirms it is not the bottleneck at our target sizes (hundreds to a few thousand neighbors); for substantially larger graphs the choice should be revisited.

7 System Limitations

Direct-link disclosure. If x and y are direct neighbors in Party 2’s graph, the paper prescribes an early “direct link” abort. We surface this as a `DirectLinkFound` exception locally but suppress the signal over the network: leaking the early-abort condition to a malicious client would be a free edge-presence oracle. The remote server therefore runs the full protocol regardless.

Manual candidate selection. The API requires the client to nominate (x, y) ; there is no built-in candidate generator. A client lacking prior knowledge of the joint graph may waste queries on uninformative pairs.

Low single-query utility. CN and Jaccard scores are useful for ranking many candidate pairs; an isolated score is not. The library supports batched calls but lacks a ranking helper.

OS support. OpenMined PSI ships prebuilt wheels for macOS and Linux only. Windows requires WSL2 or Docker.

Adamic–Adar. Not implemented. Cardinality PSI does not expose the per-element degree statistics required.

8 Future Work

8.1 Defending Against Adaptive Queries

The Demirag/Ayday protocol guarantees per-query privacy, but our system places no bound on what a client learns from an adaptive query stream. The defense is necessarily layered:

- **Per-client query budgets and rate limiting.** The lowest-hanging defense; bounds total leakage in absolute terms.
- **Differentially private answers.** Adding calibrated noise to the CN or Jaccard score trades accuracy for a formal (ϵ, δ) bound on what any adaptive sequence reveals. Composes naturally with rate limiting.
- **Query-pattern auditing.** Logging client query graphs and detecting structural signatures of reconstruction attacks is detection rather than prevention, but lets the data owner intervene before substantial leakage.
- **Authenticated, persistent client identity.** Required so the above cannot be bypassed by spinning up fresh sessions; without it, every other defense reduces to a Sybil problem.

None of these is a cryptographic guarantee on the footing of the PSI primitive, but together they are the toolkit the threat model actually requires.

8.2 Other Extensions

Heavier privacy variants. Adamic–Adar and Katz, listed as candidate measures at midterm, were not delivered: both require per-element neighbor statistics that cardinality-only PSI does not expose. Supporting them privately requires either an FHE protocol over weighted sets or a two-round PSI variant that reveals masked elements. Separately, the paper’s heavier additively-homomorphic variant hides intermediate intersection sizes from Party 1 entirely, and could be implemented on top of an existing FHE library (Microsoft SEAL, Pyfhel).

Practical extensions. The current implementation models simple, undirected, single-attribute graphs; weighted edges, directed edges, and typed nodes each require extending the data structures and protocol decomposition. The server loads from CSV at startup; backing it with Neo4j or a NetworkX-on-Postgres store would allow live updates and larger graphs. If `fastpsi` (or another OKVS+VOLE implementation) gains Python bindings, the PSI backend is small enough (~ 20 LOC) that swapping it in would be straightforward and would reduce per-query latency for large graphs.

9 Conclusion

We delivered the system summarized in Section 3.1: a Python implementation of the Demirag/Ayday two-party PPLP protocol supporting CN (3 PSI calls) and Jaccard (5 PSI calls), runnable across two machines via FastAPI and localtunnel, and accessible through a Python API, CLI, and Streamlit dashboard. We substituted OpenMined PSI for our originally planned ultra-fast PSI backend (Ling et al.) for portability. End-to-end demonstrations across four datasets confirmed correctness against a plaintext baseline; microbenchmarks show ≤ 0.6 s per query at neighborhood sizes of 1 000.

The principal systems contribution is recasting the protocol as an asymmetric server–client service (Section 3.4). The paper’s symmetric peer framing is cryptographically correct but is not how the technology will be deployed; recasting it as a long-lived data-as-a-service endpoint turns it into something a data owner can actually offer. This packaging also surfaces the most interesting open problem: the protocol guarantees per-query privacy but says nothing about adaptive query streams, and our prototype inherits this gap. Closing it—through per-client budgets, differentially private answers, query-pattern auditing, and authenticated client identity—is the direction we would prioritize over further protocol-level work. The library, documentation, and reproducible demonstrations are open source.

Acknowledgments

This project is conducted for CSDS 356/456. We thank the instructors and the authors of the cited works for the foundations on which we build, as well as the OpenMined community for maintaining a portable PSI implementation that made this prototype possible.

References

- [1] Hao Chen, Kim Laine, and Peter Rindal. 2017. Fast Private Set Intersection from Homomorphic Encryption. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. ACM, 1243–1255. doi:10.1145/3133956.3134067
- [2] Didem Demirag, Mina Namazi, Erman Ayday, and Jeremy Clark. 2022. Privacy-Preserving Link Prediction. In *Data Privacy Management, Cryptocurrencies and Blockchain Technology*. Springer International Publishing, 35–50. doi:10.1007/978-3-031-25734-6_3
- [3] Craig Gentry. 2009. Fully Homomorphic Encryption Using Ideal Lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing (STOC '09)*. ACM, 169–178.
- [4] David Liben-Nowell and Jon Kleinberg. 2007. The Link-Prediction Problem for Social Networks. *Journal of the American Society for Information Science and Technology* 58, 7 (2007), 1019–1031. doi:10.1002/asi.20591
- [5] Guowei Ling, Hao Chen, Kim Laine, and Peter Rindal. 2025. Ultra-Fast Private Set Intersection From Efficient Oblivious Key-Value Stores. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Implementation: github.com/ShallMate/fastpsi.
- [6] OpenMined. 2024. PSI: Private Set Intersection Cardinality Protocol Based on ECDH and Bloom Filters. <https://github.com/OpenMined/PSI>. Python package: `openmined-psi`.